

13. Questionnaire Design and User Parameters

13.1 What This Section Describes, and What It Does Not

The system presents a twenty-six field web form. It is not a survey instrument, and it was not administered to respondents. No questionnaire was piloted, no responses were collected, and no ethics approval was sought, because at no point did the project involve human participants. The training data was generated programmatically, as Section 12 sets out.

What follows therefore describes the *input interface* through which a live user supplies values at prediction time. Calling it a questionnaire would overstate what was built. It is the elicitation surface for the twenty-six model features, and its design problem was consequently narrow: obtain each feature from a user who is not a financial analyst, in a form the model can consume, without permitting values the model cannot represent.

13.2 Structure

The form is divided across four steps. The division mirrors the grouping of the underlying columns and exists to reduce the number of fields visible at once; twenty-six inputs on a single page would make a long and discouraging page. No usability study was conducted to verify this, and the claim is offered as a design rationale rather than an empirical finding.

Step	Group	Fields
1	Demographics	10
2	Financial profile	10
3	Lifestyle	6
4	Review and submit	—

Each step validates before the user may advance. A progress indicator across the top shows position within the sequence. The fourth step renders every entered value back for confirmation and performs no collection of its own.

13.3 Parameter Specification

Every field is constrained twice: once in the browser by a Zod schema, and once in the service by a Pydantic model. The browser check exists to give immediate feedback; the service check exists because a browser cannot be trusted. Where the two disagree, the service is authoritative, and a value that passes the browser but fails the service returns HTTP 422 with a field-level error rather than reaching the model.

Categorical fields are constrained to exact string literals. These are not free text, and this is not a cosmetic decision. The model's ordinal maps and one-hot columns were fitted on the exact spellings present in the training data, so a value of `Full-time` where the model expects `Full-Time` produces a silently different encoding. An earlier build of the frontend accepted occupation and location as free-text inputs and offered

dropdown values such as Bachelor's and Upper-Middle that appear nowhere in the training vocabulary; every such submission was rejected by validation. The current form offers only the ten permitted literal sets:

Field	Permitted values
gender	Male, Female, Other
education	High School, Graduate, Post Graduate, PhD
occupation	Salaried, Self-Employed, Business, Freelancer, Student, Retired
income_level	Low, Middle, High
marital_status	Single, Married, Divorced, Widowed
location	Urban, Semi-Urban, Rural
employment_type	Full-Time, Part-Time, Contract, Unemployed
investment_frequency	Never, Rarely, Monthly, Weekly
insurance_coverage	None, Basic, Comprehensive
shopping_frequency	Rarely, Monthly, Weekly, Daily

13.4 Numeric Bounds and the Support of the Training Data

The numeric fields carry declared bounds. Those bounds were chosen to be *permissive* — to accept any value a plausible user might report — rather than to mirror the range the model was trained on. The two differ, and the divergence is worth tabulating explicitly because it determines how the system behaves at its edges.

Field	API accepts	Training support
age	18 – 100	18 – 75
dependents	0 – 10	0 – 5
years_of_experience	0 – 50	0 – 40
savings_rate	0 – 100 %	–10 % – 60 %
emergency_fund_months	0 – 36	0 – 21
credit_score	300 – 900	300 – 900
investment_experience_years	0 – 50	0 – 15
subscription_count	0 – 30	0 – 15
online_spending_pct	0 – 100	0 – 80
luxury_spending_pct	0 – 100	0 – 50

Only `credit_score` aligns. For every other field the interface admits values the model never observed. A user aged 82, or reporting twenty-five years of investment experience, supplies a point outside the region where the training distribution has any mass, and a decision-tree ensemble asked to extrapolate beyond its training range does not extrapolate at all — it returns the value of the terminal leaf reached by the last split it can apply, which is the prediction it would have made at the boundary.

Two mitigations are in place. Values are clipped to the interquartile bounds fitted during preprocessing before any derived feature is computed, so an extreme input produces the same feature vector as a boundary input rather than an unseen one. And the model returns a probability distribution rather than a bare label, so a prediction made near the edge of the training support is at least accompanied by the confidence that reflects it. Neither mitigation makes the prediction *valid* for such a user; they only ensure it is well-defined. Section 26 records this as a limitation.

One asymmetry runs the other way. `savings_rate` is declared non-negative in the API, yet the training data contains values down to -10% , representing individuals whose expenses exceed their income. A user who is currently dis-saving cannot express that state through the form. They must enter zero, and the model will assess them as though they were breaking even.

13.5 Two Fields the Interface Handles Poorly

`insurance_coverage = "None"`. The form offers three options and validation accepts all three. The model, however, was trained on a two-level encoding, because the `None` category was destroyed when the training CSV was read, as Section 12.5 describes. A user selecting "no insurance" is therefore encoded as the training median — which corresponds to `Basic`. The system accepts the input, records it faithfully in the database, and then quietly assesses the user as though they held basic cover. This is a genuine defect, and it is a defect of the data pipeline rather than of the form.

`savings_rate` units. The field is labelled as a percentage and the API contract declares it as one, in the range 0–100. The underlying dataset column is a fraction. The conversion happens inside the inference module and is unconditional; an earlier implementation applied it only when the magnitude exceeded one, which misread a genuine entry of `0.5` — meaning half a percent — as fifty percent. The current behaviour is documented in `docs/api-contract.json`, which is the single source of truth both the frontend types and the backend schema are checked against.

13.6 Why These Twenty-Six Parameters

The parameter set was not derived from a validated psychometric instrument, and it is not claimed to be one. It reflects three groups of quantities that behavioural finance associates with risk-taking, chosen because they are things a person can plausibly report about themselves.

Demographic attributes stand in for life-stage constraints: a person with dependants carries obligations that alter their capacity to absorb loss, independent of their attitude toward it. Financial attributes capture objective position — surplus, buffer, leverage, creditworthiness. Lifestyle attributes are the least conventional inclusion and serve as observable proxies for consumption discipline, on the reasoning that discretionary spending patterns reveal preferences that a direct question about risk tolerance would not, since people are known to misreport the latter.

The distinction between *capacity* to bear risk and *tolerance* for it is the organising idea, and the parameter set deliberately weights the former. Section 3 situates this against the literature. It should be noted that the risk label in the training data is itself a function of five capacity-side variables only, so the lifestyle attributes contribute little to the model's decisions — a point the SHAP rankings in Section 21 make plain, and which is a property of the generator rather than a finding about human behaviour.

Screenshot 13.1 — *Step 1 of the assessment form (Demographics)*, showing the progress indicator and one dropdown expanded to display its permitted literal values. Place in Section 13.2.

Screenshot 13.2 — *Step 2 (Financial profile)* with a validation error triggered, for instance `credit_score` set to 250. This demonstrates the client-side bound in Section 13.3 without needing to describe it. Place there.

Screenshot 13.3 — *Step 4 (Review and submit)*, showing the full set of entered values before submission. Place at the end of Section 13.2.

Table 13.1 — *Complete parameter dictionary*. Twenty-six rows: field, group, type, UI control, permitted values or bounds, training support. Merge the two tables in Sections 13.3 and 13.4 and add the six lifestyle fields omitted there for brevity. Source the bounds from `backend/app/schemas/assessment.py`, which is authoritative. Place as an appendix.